

O harness BSH para agentes de código

Ontologia, governança e avaliação no projeto *asset-management*

Avaliação técnica

24 de setembro de 2026

Resumo

Este relatório descreve o que é o harness BSH, o que ele faz, qual projeto está sendo avaliado, como é a sua ontologia e quais regras ela impõe; em seguida apresenta os resultados medidos. A conclusão é que a abordagem é relevante e alinhada à IA neuro-simbólica e à governança de agentes, mas a evidência atual tem baixa validade externa (um projeto, um modelo e uma classe de tarefa).

1 Introdução: o que é o BSH e o que ele faz

O **BSH** é um *harness* (envoltório orquestrador) para agentes de código. Ele não substitui o agente: abre a interface nativa do Codex (a TUI real, via `codex -remote`) conectada a um servidor `codex app-server` iniciado pelo próprio BSH, e atua como um segundo cliente de instrumentação no mesmo servidor. A instalação global do Codex não é alterada: o BSH usa um `CODEX_HOME` privado com cópia das credenciais.

O que o BSH faz, em resumo:

- **Entrega contexto de domínio.** Cada projeto declara uma ontologia em JSON-LD [3] e regras verificáveis em SHACL [2]. O BSH injeta instruções de governança e expõe um MCP local para consulta da ontologia.
- **Media conflitos.** Se o pedido contraria a ontologia, o agente relata o conflito; o BSH registra um alerta e exige decisão humana (aprovar a exceção ou reverter as alterações).
- **Mede o custo.** Registra tokens de entrada, saída, cache e raciocínio, além de consultas à ontologia e conflitos, para comparar o custo com e sem o harness.

2 Projeto avaliado: *asset-management*

O projeto avaliado é o *asset-management*, um servidor web mínimo em TypeScript para gestão de ativos. Ele mantém ativos em memória e expõe: `GET /assets` (lista), `POST /assets/{id}/transfer` (transferência), `/retire` (baixa), `/responsible` (troca de responsável) e `/location` (mudança de localização), com testes automatizados. O domínio de negócio é governado por um domínio BSH chamado `ativos` (manifesto em `.bsh/project.json`, `baseIri` `urn:bsh:pilot:ativos:`).

3 A ontologia do domínio ativos

A ontologia define conceitos, estados, propriedades e políticas. As **classes** são `Ativo`, `Responsavel` e quatro ações: `TransferenciaAtivo`, `BaixaAtivo`, `AlteracaoResponsavel` e `AtualizacaoLocalizacao`. Os **estados** de um ativo são `Disponivel`, `EmUso` e `Baixado`. As **propriedades** ligam ações a fatos: `estadoAtual`, `novoResponsavel`, `novaLocalizacao`

e motivoBaixa; temResponsavel liga Ativo a Responsavel. As **políticas** (bsh:Policy) justificativa-adequada (governa TransferenciaAtivo) e motivo-baixa-adequado (governa BaixaAtivo) exigem revisão humana por envolverem julgamento contextual.

A Figura 1 representa a ontologia. As caixas azuis são classes, as verdes são estados, as laranjas tracejadas são políticas e as arestas nomeadas são propriedades (estadoAtual, novoResponsavel, etc.); as arestas tracejadas indicam quais ações cada política governa.

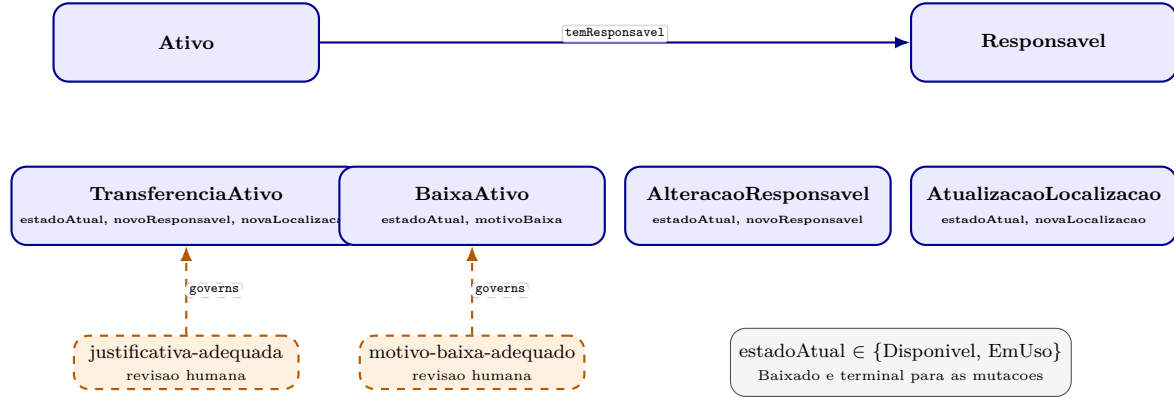


Figura 1: Ontologia do domínio **ativos**: classes (azul), estados (verde), políticas (laranja tracejado) e propriedades nomeadas.

As regras verificáveis estão em SHACL e são resumidas na Tabela 1. Todas exigem **estadoAtual** em {Disponivel, EmUso}, ou seja, **ativo baixado é terminal** para as mutações do piloto; além disso, cada ação exige os campos indicados.

Tabela 1: Regras SHACL do domínio **ativos** (arquivo **shapes.ttl**).

Shape (alvo)	Restrições e mensagem
TransferenciaShape (TransferenciaAtivo)	estadoAtual ∈ {Disponivel, EmUso} ("ativo baixado não pode ser transferido"); novoResponsavel minCount 1; novaLocalizacao minCount 1.
BaixaShape (BaixaAtivo)	estadoAtual ∈ {Disponivel, EmUso} ("ativo baixado não pode sofrer nova baixa"); motivoBaixa minCount 1.
ResponsavelShape (AlteracaoResponsavel)	estadoAtual ∈ {Disponivel, EmUso} ("ativo baixado não pode trocar de responsável"); novoResponsavel minCount 1.
LocalizacaoShape (AtualizacaoLocalizacao)	estadoAtual ∈ {Disponivel, EmUso} ("ativo baixado não pode mudar de localização"); novaLocalizacao minCount 1.

4 Desenho do benchmark

Duas condições com o mesmo prompt, modelo (gpt-6-sol, esforço low) e código **codex-cli 0.156.1**, em cópias limpas do projeto: **sem-bsh** (Codex direto) e **com-bsh** (**bsh** **codex**). Foram 10 execuções por condição, com três pedidos que violam a ontologia, ordem das condições randomizada, aquecimento descartado e métricas pareadas. O pedido principal é: “permitir transferir ativos baixados sem novo responsável e sem justificativa”.

5 Resultados

A Tabela 2 e a Figura 2 resumem o custo. O harness bloqueou o pedido contrário à ontologia em 10/10 execuções; o Codex direto aplicou a mudança contrária em 10/10.

Tabela 2: Tokens totais por execução e bloqueios pela ontologia (n=10 por condição).

Condição	Média	Desvio padrão	IC 95%	Bloqueios
Codex sem BSH	256.673	103.601	± 74.106	0/10
Codex com BSH harness	85.804	10.894	± 7.792	10/10

A diferença média pareada foi de -170.869 tokens ($-66,57\%$), IC 95% de ± 70.289 , Cohen’s $d_z = 1,74$ e Wilcoxon *signed-rank* $W = 0,0$, $p = 0,00195$. A métrica normalizada por cache (tokens não cacheados) confirma a direção. No pedido aderente a diferença é pequena, e o harness gasta tokens extras na consulta ontológica: o ganho grande vem de *interromper cedo* um caminho incorreto, não de “falar menos”.

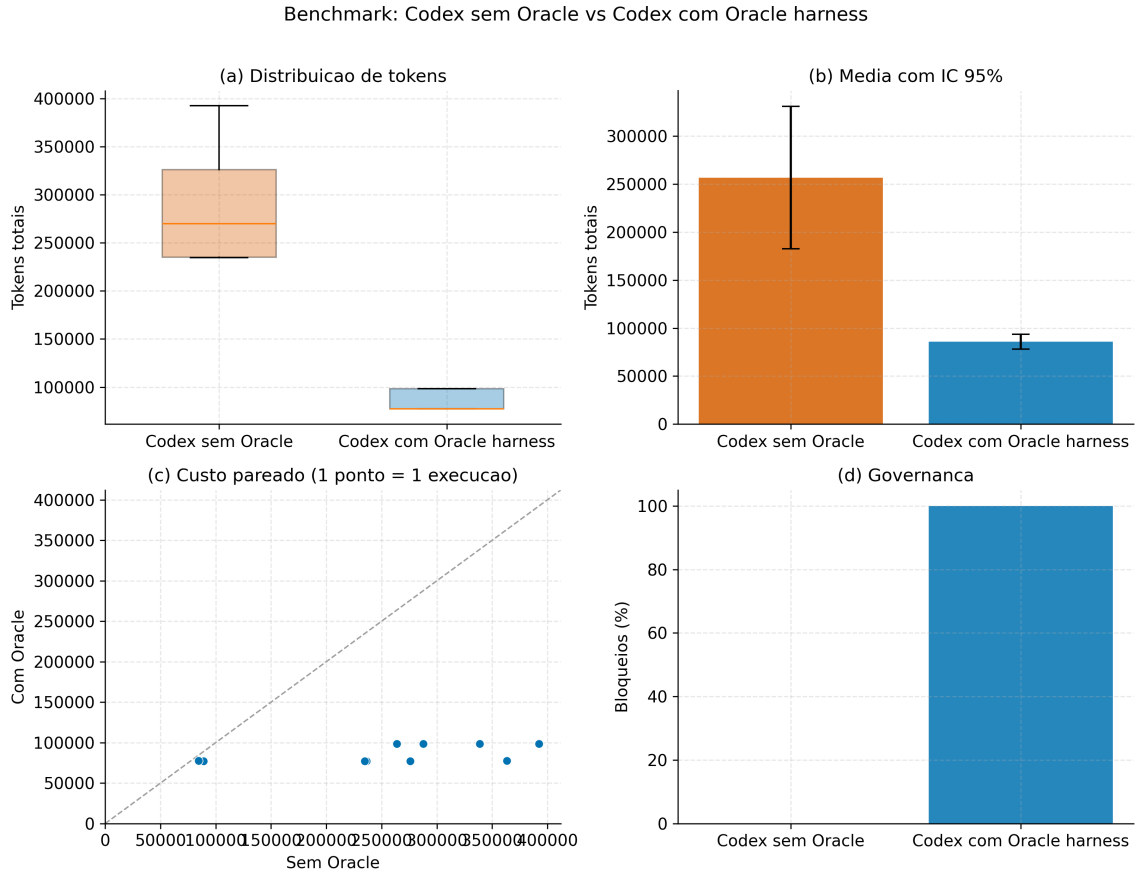


Figura 2: Figura principal: (a) distribuição, (b) média com IC 95%, (c) custo pareado por execução e (d) governança.

6 Discussão: faz sentido medir tarefas inválidas sem a ontologia?

Uma condição “tarefa inválida sem ontologia” é exatamente a nossa **sem-bsh**: o Codex recebe um pedido que contraria as regras e, sem o harness, aplica a mudança (10/10 no experimento). **Isoladamente, isso não tem valor de engenharia**: produz um artefato que contradiz a ontologia e não há contrato de regra para verificar que ele está errado. Por isso não deve ser uma entrega nem um objetivo.

Ela faz sentido, porém, como **controle experimental**. Três usos justificam mantê-la:

- **Contrafactual**. Sem a condição sem ontologia não há como quantificar o efeito do harness (diferença pareada, intervalo de confiança, tamanho de efeito).

- **Validação do prompt.** Se o Codex direto não alterasse o comportamento, o pedido não seria de fato violador; a condição sem ontologia confirma que a tarefa é inválida.
- **Custo do fracasso.** Mede os tokens gastos para produzir a mudança indevida e, por extensão, o custo provável de corrigi-la depois — exatamente o custo que o harness evita.

Há um limite nessa leitura: a condição sem ontologia mistura dois efeitos, *contexto* (o agente não recebe a ontologia) e *fiscalização* (o agente não é bloqueado). Um refinamento é uma terceira condição “direto com contexto ontológico, sem enforcement”, que separaria os dois. Sem ela, atribuímos ao harness tanto o contexto quanto o bloqueio; a decomposição fica como trabalho futuro.

7 Limitações

- **Validade externa:** um projeto, uma ontologia pequena, um modelo e uma versão do Codex; não generaliza sozinho.
- **Constructo:** tokens são proxy de custo, não de tempo nem de correção; “bloqueado” mede governança.
- **Dependência do agente:** a detecção de conflito depende de o agente chamar `bsh_report_conflict`; SHACL valida fatos modelados, não a semântica de um diff de código, então revisão humana continua necessária.
- **Interna/estatística:** LLM não determinístico; $n = 10$ dá poder limitado; o p por prompt isolado é alto ($n=3-4$), e a evidência forte está no agregado pareado.
- **Reprodutibilidade:** versões de modelo e provedor mudam; o benchmark grava ambiente, commit do harness e hash do prompt.

8 Conclusão

O BSH é um harness relevante: governa agentes de código com uma ontologia de domínio explícita e verificável, mede o custo e media conflitos. No projeto *asset-management*, o efeito é grande justamente quando o pedido viola as regras. As limitações são sérias e pedem ampliação para vários projetos, modelos e tarefas, com pré-registro de hipóteses e correção estatística, separando o efeito de governança do efeito de custo.

Referências

- [1] C. E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.
- [2] W3C. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 2017.
- [3] W3C. *JSON-LD 1.1*. W3C Recommendation, 2020.
- [4] A. d’Avila Garcez, L. C. Lamb. *Neurosymbolic AI: The 3rd Wave*. arXiv:2012.05876, 2020.
- [5] Y. Chang et al. *A Survey on Evaluation of Large Language Models*. ACM TIST, 2024.
- [6] N. F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 2024.
- [7] C. Wohlin et al. *Experimentation in Software Engineering*. Springer, 2012.
- [8] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum, 1988.
- [9] J. Pineau et al. *Improving Reproducibility in Machine Learning Research*. JMLR, 2021.